

ul> | Mixture with both. Horizontally for the base and vertically on top of it |
| Filter search results by state | Search UI add issue scope results filtering by
stateAdd issues/mergerequests filtering by state for search API | Vertically
that each merge request shipped a standalone feature |

Guidelines for proprietary and upstream contributions

Contributions in projects that have proprietary and upstream contributions will use the following guidelines to have a streamlined review.

1. The JiHu Engineering team will open two MRs:
 1. JiHu MR with all changes against the default branch in JiHu project.
 1. GitLab Inc MR with all non-jh/ changes against the default branch in GitLab Inc project.
1. The GitLab Inc MR will be reviewed by the GitLab Inc team members. Reviewers should follow the guidelines for reviewing JiHu (JH) Edition related merge requests.
1. After merging, the updates will be mirrored to JiHu project via pull mirroring and synchronized via code sync, merging into the default branch main-jh in the JiHu project.
1. The JiHu Engineering team will remove all non-jh/ changes from the JiHu MR.
1. The JiHu Engineering team will review and merge the JiHu MR in the JiHu project.
1. A scheduled pipeline every 2 hours will run in compliance-verification against the JiHu project pulling mirror to verify that there are no code difference outside of jh/ directory beside agreed difference for package.json and yarn.lock.

JiHu contribution identification

Contributions from JiHu team members are labeled with JiHu contribution label via:

- Event driven automation
- Scheduled automation as backup.

Merge request review process

1. JiHu author submits upstream merge request in draft
1. JiHu author's manager is mentioned in the merge request description
1. JiHu team will conduct a peer-review
1. Once JiHu team peer review is complete. JiHu R&D manager or maintainer will add LGTM or Looks good comment in the MR.
1. Merge request is then set to 'ready' state by JiHu team.
1. JiHu author will request a review using @gitlab-bot requestreview to identify and work on merging the MR with a merge request coach
1. The MR goes through our documented review process which includes:
 1. Code review by domain experts
 1. Review from owners of specific code files. JiHu merge request author is responsible to mention team members from list of require approvals in the MR Approvals widget. Currently for the following area:
 1. Authentication related code
 1. GitLab Security Review, which will be triggered automatically.

What approvals are required

Upstream merge requests require the same level of review and approval as all merge requests including:

- Regular code review
- Security review
- Database migration review when applicable

Upstream merge requests may require additional specific team reviews based on changed files. High impact code is identified with CODEOWNERS rules and required approvals for specific files. For example, if the merge request includes changes related to authentication or authorization, it must be approved by a Manage:Authentication and Authorization team member

What to review

- Do not review changes inside jh/:
 - If the specific change is only needed for JiHu, it should go into jh/ directory in the JiHu project repository.
- Changes outside of jh/ directory. Some examples include:
 - Features which can be added to CE/EE.
 - Refactoring which can make CE/EE code more clean or more modular.
 - Changes for prepending the classes/modules should be reviewed based on JH features based on CE or EE features.
 - Database migrations related changes should be reviewed following database migration review process.

Merge request review escalation

Please refer to our guidelines.

Release certification process

The Application Security team performs a certification of each release that includes JiHu contributions. Please see this documentation for more information about this process.

Certification issues containing a report can be found in the issue tracker of the jh-upstream-report repository.

SECTION: ceo/office-of-the-ceo/jihu-support/jihu-database-change-process.md

JiHu guidelines for database changes

For JiHu contributions that contain database migrations (for PostgreSQL), there are two routes to consider:

1. Upstream fully: Contribute the code change including database migrations upstream into GitLab

1. Upstream migrations only: Only upstream database migrations without the code change (e.g. for a JiHu proprietary feature)

The following details guidelines and background for (2): Upstream database migrations only, without the relevant code change.

For changes that are going to be fully upstreamed including code changes, we follow the regular GitLab contribution workflow and the following does not apply.

Schema changes

For structural changes, we choose to send all database migrations to the upstream project. In order to facilitate upgrade paths between GitLab CE/EE and JiHu releases, we intend to keep the database schema exactly the same across both GitLab and JiHu.

This is an extension of the pattern we already apply for GitLab CE/EE: There is only one database schema for both editions.

For JiHu-specific database objects, like columns, tables and index, we annotate those objects with a PostgreSQL comment. This will allow us to reason about the relevance of the object in a regular GitLab environment (e.g. we can ignore JiHu-specific columns there) and help us to signal-boost why the object exists in the first place (even though there is no code using it in GitLab).
See [gitlab-org/database-team/team-tasks/192](https://gitlab-org.com/database-team/team-tasks/192) for details.

We distinguish the following cases:

Column additions

A JiHu contribution may add columns to existing GitLab tables. However, those columns won't be used in the GitLab codebase. We take the following measures for clarity:

1. Annotate columns with a PostgreSQL comment to indicate the column is JiHu-specific.
1. Based on the annotations, we don't expose JiHu-specific columns to ActiveRecord in a regular GitLab environment. This will be switched off in a JiHu environment.
1. Added columns can only be set nullable (NULL) or not-null with a default NOT NULL DEFAULT x

Table/view additions

When adding new tables in a JiHu contribution, we annotate the table with a PostgreSQL comment to indicate the table is JiHu-specific. We'll have measures to prevent usage of those tables through ActiveRecord models in a regular GitLab environment.

It is fine to add foreign keys to existing tables. However, a foreign key to an existing GitLab table has to be qualified with a ON DELETE clause, which ensures cascading deletes continue to work. This is particularly important to support migrations between GitLab and JiHu (and back), where any leftover data would break cascading deletes without this clause.

Index additions

Indexes can be added to JiHu-specific tables without additional considerations.

In order to add indexes to a JiHu-specific column on a GitLab table, the index must be turned into a partial index to reduce its size on GitLab, for example:

```
sql
CREATE INDEX userdetailsphone ON userdetails (phonenumber) WHERE phonenumber IS NOT NULL
```

Adding indexes to an existing GitLab column will need to be reviewed by the database group on a case-by-case basis.

In all cases, we annotate the index with a PostgreSQL comment to indicate the index is JiHu-specific.

Optional: In order to reduce the overhead on GitLab.com, we may want to choose to ignore creating indexes that are JiHu-specific.

Additional objects

Other types of database objects, e.g. triggers, functions, extensions, etc., will need to be reviewed by the database group on a case-by-case basis.

Data migrations

A migration that mutates existing data is also merged into the upstream project. However, we'd like to avoid executing JiHu-specific data migrations in a regular GitLab environment, particularly on GitLab.com.

Any JiHu-specific data migration (including background migrations) is encouraged to target JiHu environments only and will need to go through the standard database review process.

Switching between GitLab and JiHu

With the exact same schema across GitLab and JiHu, switching from GitLab to JiHu become seamless.

However, we acknowledge that semantics for switching from JiHu to GitLab has not yet been defined. Even though we can switch back to GitLab given the same database schema, we have not yet decided how to treat existing JiHu-specific data in that situation. Once switched back to GitLab, there will be no code anymore that maintains JiHu-specific data - which might cause issues e.g. because of constraint violations or inconsistent data in case the installation is later moved back to JiHu again.

Tradeoffs/Discussion

We recognize that this pattern has the following disadvantages (with mitigations):

| Disadvantage | Mitigation |

|---|---|

| We add database objects to GitLab that are unused in the codebase. | By annotating objects, we keep track of this to reduce confusion. |

| We accept overhead for any GitLab installation, including GitLab.com, to create and maintain JiHu-specific database objects that are not strictly in use by or necessary for GitLab. | For GitLab.com, we may choose to ignore JiHu-specific indexes (we don't need to be able to upgrade to JH). |

| Always having to go through GitLab to add database migrations limits flexibility for JiHu. | We expect benefits from collaborating closely on database design in terms of knowledge exchange. |

| Code review overhead: Without accompanying code, it is often difficult to provide meaningful feedback for database design. | We ask to link relevant code changes and provide as much information as possible upfront. |

Alternative: JiHu specific migrations (not chosen)

We considered an alternative approach is to keep JiHu specific migrations separate.

JiHu may want to implement mechanics to allow for JiHu-specific migrations that are not going to be merged into the upstream project. This would allow for more flexibility and autonomy in terms of database changes, at the expense of increasing the complexity to support various upgrade paths between GitLab CE/EE and JiHu editions. Since there can be conflicts between GitLab and JiHu migrations over time much like a Git timeline, examples in (issue 161), JiHu would need to provide schema migrations specifically to each supported upgrade path from GitLab.

We didn't choose this option at this point in time to allow for quicker turnaround, less complexity and ability to seamlessly switch between GitLab and JiHu.

Considerations for database review by GitLab team members

Reviewing changes with database migrations but without the related code changes has proven to be difficult for database reviewers and maintainers.

In order to help GitLab reviewers to understand the context in more detail, we ask to have the related code change ready and reviewed, before asking for a database review.

The related code change should be linked from the merge request, along with any useful background information.

Otherwise we follow the same process as outlined in How to prepare the merge request for database review.

Sending merge requests with database migrations from a fork

In order to increase the review efficiency when creating a merge request from a fork, we suggest to configure the DANGERGITLABAPITOKEN in the forked project. This will allow to run Danger, which is going to suggest relevant reviewers among other helpful content relevant for review.

Questions?

Please engage with the database group for any questions and support.

SECTION: ceo/office-of-the-ceo/jihu-support/jihu-security-review-process.md

Security review process for JiHu contributions

In order to make sure that a release with JiHu contributions can be certified, members of the AppSec team need to perform a security review of any JiHu contribution and explicitly approve them. The JiHu team will be contributing code to a number of repositories. These contributions will be auto labeled with the JiHu contribution label.

mermaid
graph TD

```
A[JiHu Contribution Opened] -->|First merge request review approval| B(AppSec approval requested)
B --> C["AppSec approves with MR approval and ~sec-planning::complete"]
C --> D[Automated commit AppSec approval comment]
D --> E{Merge requests is updated}
E -->|Yes| B
E -->|No| F[Merge request can be merged]
```

Who can perform a security review of a JiHu contribution

Any member of the AppSec or Federal AppSec team is eligible to perform a security review of a JiHu contribution.

When to request a review

An automated comment pings the AppSec team after the MR receives its first approval. Team members do not need to manually @-mention AppSec.

Determining who will perform a security review of a JiHu contribution

When the AppSec team is pinged on a JiHu contribution, it will typically be first seen by the AppSec engineer on Triage (mentions and issues) Rotation. This person should:

1. Ping the stable counterpart for the relevant part of the codebase and ask them to perform the review
 - If the change is small or easy to review, the AppSec engineer on triage can do the review themselves and @-mention the stable counterpart for visibility
1. If the stable counterpart is unavailable or unassigned, the AppSec engineer on Triage Rotation can perform the review
1. Alternatively, they can ask that someone else review it by linking to the MR in the sec-

appsec Slack channel and asking for a reviewer to volunteer

Security review workflow for JiHu contributions

When performing the security review of a JiHu contribution, the reviewer needs to:

1. Perform a security review of the merge request
 - Make any comments or ask any clarifications necessary to complete the review
 - Look to make sure that the code does not introduce any new vulnerabilities
1. If the merge request looks acceptable:
 - Make a comment that uses the /approve quick action
 - Approval will be confirmed with an automated approval comment
 - Apply the correct AppSec labels:
/label ~"AppSecWorkType::JihuMRreview" ~AppSecWeight::<update> ~"Application Security Team" ~AppSecWorkflow::complete"
/milestone %<update>
1. If the merge request does not look acceptable at this time, and/or introduces new vulnerabilities, and/or the AppSec team is waiting on answers from the engineer:
 - Apply the sec-planning::pending followup label
 - If possible, work with the creator to make it secure. Follow the steps above once it looks acceptable
 - If inherently unacceptable or a wider discussion is needed, express any concerns and work with the creator and the relevant product and engineering teams to move forward

Security final commit approval

AppSec approvals are revoked when subsequent changes are added to the merge requests and requires a re-review from AppSec before merging. The process is as follows:

1. ~sec-planning::complete will be revoked when the MR is updated (additional commits or rebase) and the original AppSec approver will have a request to re-review and approve.
1. Use the "Compare Versions" feature to see what changed between the AppSec-approved version and the latest version. The automated approval comment contains the approved version's SHA hash.
1. Use the suggested quick action to unapprove and re-approve after re-reviewing.

Merging of gitlab-org/gitlab ~JiHu contribution MRs is blocked if the ~sec-planning::complete label is missing from the merge request, since the verify-approvals job will fail.

SECTION: ceo/office-of-the-ceo/jihu-support/jihu-validation-pipelines.md

JiHu validation pipelines

In order to make sure our regular development does not break JiHu's pipelines, we run JiHu validation pipelines in merge requests which we consider it's more likely to break JiHu's pipelines.

Those pipelines are currently allowed to fail, and they should not block

any merge requests from being merged.

To revisit this policy, checkout: Disallow as-if-jh pipeline to fail

When and where we run it

We run the validation pipelines when we detected that the changes is more likely to break JiHu's pipeline. The detection is automated and we can also apply label ~"pipeline:run-as-if-jh" to force it if not detected.

Job start-as-if-jh will trigger a cross project downstream pipeline in the GitLab JH validation project.

For how the detection works and technical details about how we run this validation pipeline, checkout:

As-if-JH cross project downstream pipeline

What to do when the validation pipeline failed

In the case when the validation pipeline failed, follow this workflow:

- Check if the failure is known by searching for existing issues or merge requests:
 - Issues labeled with ~"JiHu Broken Pipeline"
 - Merge requests labeled with ~"JiHu Broken Pipeline"
 - JiHu's issues labeled with ~"main-jh broken"
 - JiHu's merge requests labeled with ~"main-jh broken"
- Check if JiHu's pipelines are broken

If any related items can be found, feel free to drop a reference in the issue or merge request, and move on.

If nothing can be found, follow this workflow:

- Create an issue and notify JiHu for the potential breakage. You can contact @qianzhangxa
- Add ~"JiHu impacted" label to the merge request so we can track this.
- (Bonus) Investigate the failure and follow up accordingly. This can be telling JiHu why and how the failure can happen, or what they need to do or what they need to change. To better support them, figuring out a way so they can more easily follow up is encouraged.

SECTION: ceo/office-of-the-ceo/jihu-support/release-certification.md

Process for Release Certification

Every release with JiHu contributions needs to be certified by a member of the Federal Application Security team.

This is required to satisfy PubSec/FedRamp requirements and to handle JiHu's merge request contributions to GitLab Inc repositories.

This process involves ensuring that each JiHu contribution included in the release has been reviewed and approved by an Application Security team member and posting a comment on the relevant release task issue that no new vulnerabilities have been identified in the code being released.

Who can certify a release

As per PubSec/FedRamp requirements, only United States citizens on United States soil are eligible to certify a release.

This means that only members of the Federal AppSec team are eligible to perform the certification of the release.

Note that any member of the Application Security team may review and approve a JiHu contribution,

but only Federal AppSec team members can certify the release.

Members of the Federal AppSec team are responsible for certifying monthly releases. Their assignment is managed by automation. These assignments will also need to be reflected on the Release Managers page, under the AppSec Release Certification column.

Certification process

The certification process can only begin once it is completely certain that no new JiHu contributions will be included in the release. The recommended time to begin is during the day before the release section of the release task, which specifies that no new code can be added to the release.

Once it is certain that no new JiHu contributions will be added, follow the steps below:

1. On the release date of each month the `jh-upstream-report` repository should run a scheduled pipeline that automatically creates the release certification issue. This will create an issue in the `jh-upstream-report` issue tracker with a checklist containing each JiHu contribution associated with the upcoming release. If something went wrong, the release certification tools script may need to be ran manually by following the directions in the `README.md`

1. Verify that every JiHu contribution going into the release is on this list. This can be done by looking at the status report repository information in addition to searching for the JiHu Contribution labels in each repository (the certification issue should have a link available). Be sure to look for both open and closed merge requests. The most likely reason a MR would be in the release but not in the checklist is the appropriate milestone had not been set for it.

1. For each JiHu contribution on the checklist:

- Look at the merge request and verify that an AppSec reviewer has indicated it has been reviewed and is acceptable

- If a merge request did not receive an Application Security review, perform a review and apply the appropriate labels

- If a merge request did not receive an Application Security review, consider gently reminding the person who merged it that the Application Security team is supposed to be reviewing these before a merge happens

- Occasionally, a review will have happened and the label indicating that it was performed did not get applied -- in these situations, go ahead and add the label to the merge request

- Some merge requests will still be in the Opened state, which means they were not part of

the release but they are still associated with the release milestone -- in these situations, the checkbox can just be checked

- Time allowing, briefly review the changes to re-confirm they are acceptable
- Mark the corresponding checkbox in the release certification issue to indicate the MR has been confirmed acceptable

1. There is a section of repositories that will require manual review. Typically these repositories cannot be included in the automation because the automation uses a project access token that cannot view private repositories or repositories that have the merge request section set as private. For each of these repositories:

- View the merge request section of each repository and look for any JiHu contributions that were merged
- Be aware that these merge requests may not be connected to a milestone or a particular release, so it may be necessary to sort Merged merge requests by Merged date and look for JiHu contributions made in the last month
- Some of the repositories listed may have a note with information to consider when performing the review
- If any JiHu contributions are found, remove the default No JiHu contribution merge requests were included checkbox and add new checkboxes for each JiHu contribution that were found
- If no JiHu contributions are found, check the No JiHu contribution merge requests were included checkbox
- When finished reviewing the repository for JiHu contributions, check the This repository was manually reviewed checkbox to indicate a review of that repository was completed

1. When absolutely sure that all JiHu contributions in the release have been reviewed by an AppSec team member:

- Copy and paste the boilerplate comment generated at the bottom of the release certification issue into a comment on the release task issue
- Make a comment on the release certification issue indicating that the process is complete and linking to the certification comment made on the release task issue
- Close the release certification issue

Contributions that introduce vulnerabilities

If a potential S1 or S2 vulnerability is identified in any of the contributions:

1. Comment on the release certification issue with a link to the MR and a description of the vulnerability

1. Post a message in the sec-appsec Slack channel with a link to the comment

1. Ping the appropriate release manager from the delivery team and work with them to remove the MRs from the release

- If the vulnerable code is removed, continue with the certification process
- If the vulnerable code cannot be removed, follow the When a release cannot be certified steps listed below

When a release cannot be certified

In some rare situations, a member of the Federal AppSec team may choose not to certify a release. This may happen because a known vulnerability is included in the release and it cannot be removed or other situations including (more examples TBD)

In the event that the release cannot be certified:

1. Write a comment on the release certification issue briefly explaining the reasoning behind choosing not to certify release

1. Ping security leadership (@juliedavila and @jritchey) on the release certification issue in a reply to the comment made above

1. Announce in the sec-appsec Slack channel that the release cannot be certified, link to the release certification issue, and @ mention @appsec-leadership

SECTION: ceo/office-of-the-ceo/readmes/_index.md

SECTION: ceo/office-of-the-ceo/readmes/cynthia-ng.md

This page is meant to provide a bit of information about me and how I work, in the hopes of building trust and giving others an idea of what it might be like to work with me.

It's also meant to make it easy to find and talk to me.

A little about me

I'm often described as someone who is a bridge between the technical side and the user-facing side.

This is actually my first "tech job", and first startup. I come from a mostly public services background, having been a teacher and librarian, with a significant focus on technology and accessibility.

I used to be a manager, and I clearly still have a fairly manager-like outlook on a lot of work, but I purposely went back to being an individual contributor, especially to focus on project work, and collaborate with other teams.

I believe I've come to really embrace the GitLab Values, but it initially took me months to really internalize, and I'm still working on excelling in some areas.

Aside from work, you'll hear me talk about all sorts of things, but namely books, movies, food, birds (and birding), puzzles, Lego, Canada, languages, and plushies (check out the pets page)!

My assumptions

Being at GitLab, I generally assume that people are following or embody the GitLab Values. In particular, the assumptions I have about others tend to be the same things people say about

me:

- You'll do what you say you will.
- You'll share, especially by asking for help.
- You're direct.
- You'll provide feedback if you have any.

Availability and communication preferences

I prefer asynchronous written communication, but also love to chat and get to know team members.

- Not urgent? Mention me on an issue.
- Urgent or time sensitive? Ping me on Slack.

I'm a big fan of donut/coffee chats, and learning more about others.

If you would like a synchronous meeting, feel free to find a free time on my calendar and send an invite.

If it's outside my working hours, you can still send an invite to propose a time.

Finding me

You will find me regularly in team meetings, and in various donut channels.

I also participate in TMRGs, namely Women+, and Global voices.

Ways to connect with me:

- Slack: arty-chan
- GitLab (internal)
- Support Driven
- Write the Docs
- GitLab.com: @cynthia
- Personal Blog: cynthiang.ca
- Digital card (with links to social accounts): [About.Me](#)

Expertise

- Technical topics (as of 2023-11-01):
 - Authentication/Authorization (SAML, SCIM, permissions, etc.)
 - CI
 - CustomersDot, especially CDot & GitLab.com connections, and console
 - SaaS, including GitLab.com console
 - User account issues, including intersection with legal/security
- Using GitLab for work
- Workflow/Processes
- Documentation

- Training
- UX/Accessibility
- Cross-team communication
- Tracking and prioritizing work, including for performance reviews and promotions
- Applying GitLab values, including TeamOps
- Mentoring/coaching and being a non-manager leader

Strengths and weaknesses

- I take ownership of my work and focus on results with a bias for action. At best, this means I deliver, and get a lot done.

At worst, I lose focus of the human element, seem very certain of an opinion, and/or I'm unintentionally discouraging or not giving others the opportunities to contribute.

- What to do at its worst: Tell me you'd like to contribute or remind me to think of the human impact.

Let me know that you might disagree with my idea, as many of them may sound like strong opinions even though they are not.

Because I don't do this consciously, I will never get mad at someone for letting me know.

- I'm direct. At best, it means I'm being transparent and providing feedback.

At worst, the feedback will read as criticizing, forgetting the "kind" part of directness.

- What to do at its worst: Give me feedback on what didn't work for you.

Level of directness varies by country/culture and I recognize that I'm on the direct end, so it's okay to remind me of that.

- I reprioritize constantly. At best, this means that important and impactful things get done quickly.

At worst, it means that I miss a deliverable that you were waiting for.

- What to do at its worst: Give me a deadline for what you need. Check in on progress if you need to.

If what I'm prioritizing doesn't match your priorities, I should be able to articulate what's blocking me.

- I like to share ideas and solve problems out loud. At best, this means great brainstorming sessions.

At worst, I don't think something through before I blurt it out, it might seem dismissive of other's ideas, or dominate the discussion.

- What to do at its worst: I've been working on doing this less in group settings, but tell me if I'm talking too much.

- I want to consider all aspects. At best, this means lowering risk, and making people feel included.

At worst, I make things seem more complex than they need to be, or I cause information overload.

- What to do at its worst: Ask me to summarize and simplify.

- I'm great at making connections. At best, I get to know a lot of people and things, and connect them.

At worst, it means I'm a "jack of all trades, master of none" kind of person.

- What to do at its worst: Block off some time in my calendar if you'd like me to dig deeper into something (with you).

SECTION: ceo/office-of-the-ceo/readmes/streas.md

Stella's README

Stella Treas, Chief of Staff to the CEO

This page is intended to help others understand what it might be like to work with me, especially people who haven't worked with me before.

It's also a well-intentioned effort at building some trust by being intentionally vulnerable, and to share my ideas of a good working relationship to reduce the anxiety of people who might be on my team.

Please feel free to contribute to this page by opening a merge request.

Related pages

- [GitLab](#)
- [LinkedIn profile](#)

About me

- I'm from Southern California, but I left home when I was 18. I've since lived in New Hampshire, Barcelona, New York, Boston, Washington, D.C., London, Rwanda, and San Francisco.
- Before having children, my passion was travel. I've been to over 70 countries and aggressively checked off most of the items that had been on a bucket list that I created when I was 18 (swam with dolphins, lived abroad, jumped from a plane, visited mountain gorillas, went on a tiger safari, etc.).
- I care about a number of social causes. Before getting into technology, I worked for a number of nonprofits in various roles.
- My husband and I met on a trip to climb Half Dome in Yosemite.
- I have 2 young children who are the center of my world outside of work. While I look forward to the day when I can get an uninterrupted night of sleep, they are pretty awesome.
- Starting the day with hot coffee brings me great happiness. I grow increasingly articulate as I work my way through the first cup.

My personal values

1. Impact. I care deeply about the personal value that I deliver within whatever role or project that I'm part of. I am happiest when I'm working on big projects and initiatives that help to meaningfully advance the company.
1. Integrity. There are many ways to have an impact. While integrity can mean many things, to me, it is the "how" in getting things done. Integrity means delivering results in a way that optimizes for the company as a whole. GitLab's values of collaboration and transparency fall into my definition. This is why I choose to work in values oriented and non-political companies where doing good work in a collaborative and efficient way is a priority.
1. Ownership. I own my work. This means delivering on my commitments and against my goals. When it looks like I can't for some reason, I quickly escalate, so I am aligned with others on the new timing or direction. I expect this ownership from others. I don't like to micromanage and find that it is less necessary when folks are owning their work.
1. Community. I like to invest in communities and community creation, both at work and in my personal life. I believe that positive professional and social connections make work and life

more enjoyable.

My superpowers

1. Bias for action. If I believe that something needs to be owned and I don't see anyone owning it, I'll try to step in to make sure that it gets done.
1. Problem solving. I am a problem solver. I enjoy tackling tough challenges and connecting dots. I am good at seeing trends across a company and can think both at a high level and in the details.
1. Keeping balls in the air. I rarely drop balls. If I ever do, you can be assured that my disappointment in myself is far greater than any disappointment that you have in me.
1. Collaboration. I enjoy working with others and collaborating to get to a good outcome. In my role, this is something that I need to do well.

My weaknesses

1. Always on problem solving. Depending on the situation, this can be a blessing or a curse. If you have a problem and you want a listener or aren't looking for new ideas, proactively let me know, so I can engage you with you in the right way.
1. Aversion to politics. I get demotivated when I suspect that folks are prioritizing themselves or their teams over the greater interests of a company.
1. Breadth over specialization. I am a generalist. Against a lot of advice, I've built a career around being decent at most things but not an expert in any. I overcome the downside of this through having a growth mindset and being able to ramp quickly in new areas.
1. Perfectionism. I care about my work to the degree that I can push myself to the point of running on fumes. I'm better at pushing my team toward work life balance, but I'm not always a great role model.

My expectations of everyone (including myself)

1. Own your work. Do the work that you commit to doing. If you can't for some reason, share immediately and figure out how to get it done if it is still the right thing to do.
1. Work according to GitLab's values. Our shared work values are a gift. They help increase trust and align expectations in how we work and what we can expect from each other. If you ever feel that I'm not working in a way that is conducive with GitLab values, let me know, so I can learn from your observation.
1. Give feedback. Feedback is a gift. Knowing where I am now and how to continue to grow is important to me. I also believe that helping people understand their current performance and opportunities is a gift to others. If you have feedback, don't hold back and give it in a timely manner. I will commit to doing the same.
1. Ask for greater engagement. I try to adjust my leadership and management style for each person, but I am not a micromanager. If you need more coaching and greater involvement, let me know. I will do the same.

My role

I am the Chief of Staff to the CEO at GitLab. Here is what I do in 12 words or less that is unique to my role:

Enable the CEO to achieve even greater results for GitLab.

Communication preferences

1. I work largely within Slack and issues. If it is a busy day, I may not get to email until the evening.

1. Since I work with folks in China and have young kids, I sometimes don't get online until after 8:00 am unless there is a specific reason to do so. During these times, I often read meeting notes instead of attending synchronously.

1. When inviting me to a meeting, please make it clear in the invite or meeting agenda why my participation is important. This helps me manage my time appropriately among competing priorities.

1. Please try to send meeting invites 2+ days ahead of when you'd like to meet, unless we've agreed to otherwise. It helps me better plan my work and schedule ahead of time.

1. My job has a broad scope, so I'm constantly prioritizing a long list of work and context switching. I do my own prioritization, so it may not align with yours. If something is time sensitive, please clearly let me know.

1. In Slack, I am emoji happy, because they are efficient to post, and I believe that an emoji can be superior to 100 words.

Expectations for leadership - What I need from you. What you will get from me

This mirrors my "expectations for everyone (including myself)." Point me toward where I can be of most value and be open with feedback. I commit to earning and maintaining your trust. If I don't, this should be part of your feedback.

Frequently asked questions from prior roles

1. How would you describe the work that you do?

I think of myself as an operator, and I care about overall business and organizational health. My top priority is enabling the CEO as this can have a great, positive impact on the organization. I also, in collaboration with the broader OCEO Team and with the blessing of the CEO, identify and address business gaps and pursue business opportunities.

1. You are a sociology major and worked in nonprofits. How did you wind up in tech and doing what you are doing?

Sociology taught me to think holistically about organizational effectiveness. This is a valuable thing to be thinking about in an operating role. In my nonprofit roles, I loved leading key strategy and operations activities. I also had to be scrappy. I've been able to bring what I've learned into tech operations.

1. Why are you no longer in customer success?

I love driving results for customers. A business cannot be successful when customers are not getting value. That said, I found that my unique contribution was in operationalizing and scaling a lot of what my customer success team did. And, then doing this at a higher level across functions.

How you can help me

- Reach out when you see opportunities for me to do something better and drive greater results. I appreciate any and all feedback. I've been in meetings in which team members have reached out through Slack immediately after to share their impressions--positive or constructive. In all cases, I see it as a gift. I don't want feedback to wait until a 360.

What I assume about others

- They also care about their work and are committed to operating in a way that reflects GitLab values.

What I want to learn

- How to effectively scale a company with strong and unique values

Personality tests

2022 TeamSight:

- 1. Ideas: 3 (how that would work here)
- 1. Emotion: 3 (I want fair)
- 1. Action: 5 (directs projects)
- 1. Order: 3 (available trade-offs)

2016 Five Dynamics by Simpli5: Effortless across 5 dimensions. Spikes in Examine and Execute.

2007 Meyers Briggs: ENTP. Since self diagnosed as an E N T P/J

- 1. Extrovert
- 1. Intuition
- 1. Thinking
- 1. Prospecting / Judging

SECTION: ceo/office-of-the-ceo/readmes/yyugitlab.md

{{% include-readme username="yyugitlab" trim=6 %}}

SECTION: communication/_index.md

We're an all-remote company that allows people to work from almost anywhere in the world. We hire great people regardless of where they live, but with GitLab team members across more than 60 countries, it's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

To accomplish this, we use **asynchronous communication** as a starting point and stay as open and transparent as we can by communicating through public issues, merge requests, and Slack channels.

We also place an emphasis on ensuring that conclusions of offline conversations are written down.

When we go back and forth three times, we jump on a synchronous video call.

We communicate respectfully and professionally at all times.

Effective & Responsible Communication Guidelines

1. Assume Positive Intent. Always begin with a position of positivity and grace.
1. Kindness Matters. You are looking at a screen, but you are really talking to a person. If you wouldn't say it to a person's face, do not send it to them in a text message.
1. Express Your Thoughts Responsibly and Inclusively. We live in different locations and often have very different perspectives. We want to know your thoughts, opinions, and feelings on things. We also ask you to consider the guidelines around communicating potentially sensitive topics.
1. Own It. If you say it or type it, own it. If it hurts the company or an individual, even unintentionally, we encourage you to look at things from other points of view and apologize easily.
1. Be a Role Model of our Values.
1. Feedback is Essential. It is difficult to know what is appropriate in every one of our team members 60+ countries. We encourage team members to give feedback and receive feedback in a considerate way.
1. Do not underestimate a 1:1. Asynchronous communication (e.g., via text) is helpful and necessary. In some cases (e.g., to clarify misunderstandings) it can be much more effective to jump on a Zoom video call.
1. Always Adhere to our Anti-Harassment Policy and GitLab Code of Business Conduct and Ethics. Everyone should be comfortable in their work environment.
1. Focus on what we can directly influence. There are many factors we can't directly influence and we should avoid spending time discussing those things. For example, we don't talk about our market capitalization because aspects of this are out of our control. Instead, we should focus on our KPIs and growing annual recurring revenue.
1. Commit to active and effective listening.

Embracing asynchronous communication and learning to use it effectively requires a mental shift. This can feel unusual or even uncomfortable for those who come from a colocated environment, where in-person meetings and communiques are the norm. Learn more about mastering the use of the written word in an all-remote setting.

Everyone is a moderator

If you see something that concerns you in Slack, Issues, Merge Requests, Video, Emails or any other forum, we encourage you to respectfully say something directly to the individual in a 1:1 format.

If there is an issue to raise regarding someone's communication or conduct, team members should follow the process for raising communication concerns by sharing their concern with their manager or, if preferred, email Team Member Relations (teammemberrelations@gitlab.com) directly.

Asynchronous communication

In an all-remote setting, where team members are empowered to live and work where they're most fulfilled, mastering asynchronous workflows is vital to avoiding dysfunction and enjoying outsized efficiencies and lifestyle flexibility. Asynchronous communication is the art of communicating and moving projects forward without the need for additional stakeholders to be available at the same time your communicate is sent.

To learn more on when to use asynchronous and synchronous communication, examples of async workflows in practice at GitLab, core async behaviors, and to take an async knowledge assessment, visit GitLab's guide to embracing asynchronous communication.

Communicate directly

When working on a problem or issue, communicate directly with the people you need support from rather than working through reporting lines. Direct communication with the people you need to collaborate with is more efficient than working through your manager, their manager, or another intermediary.

Escalate to management if you are not getting the support you need. Remember that everyone is a manager of one and they might have to complete their own assignments and inform the reporting lines.

Communicating Potentially GitLab Sensitive Topics

(This guidance supplements and overlaps with GitLab's SAFE Framework, the guidance on the use of the internal handbook, and the additional guidance on this page. We ask our team members to consider the factors below in their communication.)

As GitLab matures, we want to continue to foster discussion while evolving our communication guidelines so that topics that are potentially GitLab sensitive are discussed in appropriate forums. This is particularly relevant as team members heavily leverage async modes of communication including merge requests, issues and epics, and in Slack communication.

Words have impact long after they are written, and even when you're communicating internally, the manner in which you speak with one another should be viewed through an external lens. For additional information, please review our Guidelines for communicating effectively and responsibly through text.

Confidentiality levels

At GitLab, we are public by default, but some information is classified as internal or limited access. Please see the confidentiality levels handbook page for details on this.

Examples of Potentially GitLab Sensitive Topics

1. Team member data (individual performance, start dates, departures)
1. Violations, or potential violations, of policies and/or local rules and regulations
1. Customer or partner information (logos, trademarks, spend)
1. Material nonpublic information

The above examples overlap with the GitLab's SAFE Framework examples. We recommend you to further review that page for more information and context.

What are the risks?

1. Legal risk: These are the risks that arise from regulations and laws that govern GitLab or the market in which it operates. This includes, but is not limited to, content that would compromise a GitLab team member, customer, or user's personal data and/or privacy.

1. Morale risk: Raising GitLab sensitive topics that may be misinterpreted without the opportunity to ask clarifying questions can create risk to team culture and/or morale.

1. PR risk: Remember that anything you document could ultimately be shared/viewed externally. Consider that a discussion in a public MR or issue is a demonstration of our values to those outside of GitLab who are looking to learn more about how we collaborate.

We encourage communicating risks to GitLab, its team members, or customers in a synchronous 1:1 setting.

Communications Champions

Where possible, a group of Communications Champions, made up of global team members and people managers, will be given a preview of company-wide changes to provide feedback, so that team member perspectives have been taken into account.

Communications Champion cohorts

We'll introduce two cohorts for FY25:

1. All-company cohort: 10 team members who review all-company changes/messaging; this group consists of global team members at all levels.

1. People manager cohort: 7-10 team members who reviews people-manager specific changes/messaging; this group consists of global people managers.

Team member participation

Each team member will participate for two quarters/6 months. Should a team member no longer be able to participate during their cohort; they can be backfilled.

We'll engage bi-weekly with planned or urgent information seeking.

We'll create a net-new slack channel for each group:

Naming convention: comms-champions-fy25-a

- a is FY25 Q1/Q2 all-company
- b is FY25 Q1/Q2 people manager
- c is FY25 Q3/Q4 all-company
- d is FY25 Q3/Q4 people manager

Nominations and selection

People managers will nominate team members through a google form. After nominations, PBPs will review the list to ensure all are in good standing and collaborate on potential participants.

People Comms & Engagement will make final selections.

Determining Which Communication Forum To Use

The table below outlines an overview of different communication forums at GitLab, and the considerations team members should think through related to potentially GitLab Sensitive topics when determining which forum to leverage.

Communication Forum	When To Utilize
-----	-----
Use the internal Handbook aligned with the guidelines	When you want to document information for team members that is internal-only, including not public and not limited access
External MR/issue (not confidential)	For discussion and collaboration when there is no risk suspected or identified (directly or later) and it doesn't fall into not public category
Internal MR/ confidential issue	For discussion and collaboration when there is no risk suspected or identified but adhering to things that are internal-only, not public, SAFE Framework/General Communication Guidelines
Your Manager, DRI and/or Legal	For discussion and collaboration when in doubt about potential risk and you want to review if there's a potential risk
DRI	For discussion and collaboration when there's a risk suspected or identified, communicate directly with the DRI verbally via Zoom. Examples include issues where Limited Access applies or it covers a change to People Process/policy.
People Group (your People Business Partner or Team Member Relations)	For discussion and collaboration when there's a risk suspected or identified, a policy violation, and/or it's a private matter
GitLab's Whistleblower policy	For flagging a situation that is a violation as set out in the policy

When in doubt, you can reach out to your People Business Partner and/or your leadership team directly.

Organization code names

Please see our Project names section.

Internal communication

Internal communication is any work related communication at a company.

Internal Communication includes conversations between team members, wider team discussions, or internal announcements.

At GitLab, everyone can contribute to the effectiveness of Internal Communications to support aspects of GitLab culture, such as intentional transparency and engaging people in open dialogue.

Since we believe that all team members must be Managers of One, most communication is handled by the relevant group, but we know that some communications are more sensitive and contentious than others.

In those cases, the DRIs may want to engage the Internal Communications function.

Top tips and best practices

1. All written communication happens in English, even when sent one on one, because sometimes

you need to forward an email or chat.

1. Use asynchronous communication when possible: merge requests (preferred) or issues. Announcements happen on the appropriate Slack channels and people should be able to do their work without getting interrupted by chat.

1. Discussion in issues or Merge Requests is preferred over everything else. If you need a response urgently, you can Slack someone with a link to your comment on an issue or merge request, asking them to respond there, however be aware that they still may not see it straight away. See Slack for more.

1. If you choose to email instead of chat it is OK to send an internal email that contains only a short message, similar as you would use in chat.

1. You are not expected to be available all the time. There is no expectation to respond to messages outside of your planned working hours.

1. Sometimes synchronous communication is the better option, but do not default to it. For example, a video call can clear things up quickly when you are blocked. See the guidelines on video chats for more detail.

1. It is very OK to ask as many questions as you have. Please ask them so many people can answer them and many people see the answer. Use issues or public chat channels (like questions) instead of direct messages or one-on-one emails. If you have researched in the handbook and could not find the answer or need clarity, include the handbook link you were reviewing and state "while looking in the handbook I could not find x,y,z".

1. If you send a handbook link to someone as an answer to a question, consider adding some context, especially if they are new to GitLab. It's great that we often have the answer documented, but it's not always easy to find if you are a new team member.

1. If the answer to a question isn't documented, please immediately make a merge request to add it to the handbook in a place you have looked for it. It is great for the person who answered the question to see you leading by example to ensure that question only needs to be answered once. A merge request is the best way to say thanks for help.

1. If you mention something (a merge request, issue, commit, webpage, comment, etc.) please include a link to it.

1. All company data should be shareable by default. Do not use a local text file, but rather leave comments on an issue.

1. When someone asks something, give back a deadline or that you did it. Answers like: 'will do', 'OK', 'it is on my todo list' are not helpful. If it is small it's better to spend 2 minutes and do the tasks so the other person can mentally forget about it. If it is large you need to figure out when you'll do it, by returning that information the other person might decide to solve it in another way if it takes too long.

1. It is OK to bring an issue to someone's attention with a CC ("cc @user"), but CCs alone are not enough if specific action is needed from someone. The mentioned user may read the issue and take no further action. If you need something, please explicitly communicate your need along with @ mentioning who you need it from.

1. Avoid creating private groups for internal discussions:

1. It's disturbing (all users in the group get notified for each message).

1. It's not searchable.

1. It's not shareable: there is no way to add people in the group (and this often leads to multiple groups creation).

1. They don't have a subject, so everyone has to remember the topic of each private group based on the participants, or open the group again to read the content.

1. History is lost when leaving the group or after 90 days.

1. It is perfectly fine to create a channel, even for a single customer meeting. These channels should be named "a\<customer-name>-internal" to indicate their "internal" nature (not shared

with customers).

1. Use low-context communications by being explicit in your communications. We are a remote-only company, located all over the world. Provide as much context as possible to avoid confusion. Relatedly, we use ubiquitous language for communication efficiency.

1. When discussing concepts, be careful not to lean too much into hypotheticals. There is a tipping point in which it decreases value and no longer becomes constructive at helping everyone come into a unified decision.

1. Consult our tips for better writing.

Internal Communication Channel Guidance

| What/When|The GitLab Handbook | The Loop (comming soon) | Slack | Email/Newsletters | Meetings |

| ----- | ----- | ----- | ----- | -----|

| Primary Purpose | Transparency around GitLab procedures, documenting information handbook-first | Connect team members with news, information, and stories | Real-time collaboration and quick communications | Formal communications, select company updates, and consolidated updates to targeted audiences | Synchronous discussions and collaboration |

| Best For | 1.) Sharing processes and/or procedures valuable to customers, community, or candidates 2.) Creating external transparency (core values, mission, and vision) 3.)

Documenting workflows and standardized processes | 1.) Finding/sharing company information, announcements, and events 2.) Learning/sharing programs and initiatives 3.) Accessing personalized and relevant content 4.) Team member storytelling, connection, and engagement 5.)

Team showcases and wins| 1.) Immediate discussions 2.) Quick questions 3.) Team coordination 4.) Time-sensitive, informal updates 5.) Instantaneous feedback/communication 6.) Channel-

specific discussions | 1.) Critical and/or formal announcements 2.) Newsletters with targeted updates 3.) External communications (customers, vendors, etc.) 4.) Items requiring official email notification 5.) Informing targeted audiences 6.) Select leadership communications | 1.)

Complex discussions 2.) Decision-making sessions 3.) Relationship building 4.) Project kickoffs/reviews 5.) Interactive sessions 6.) Ideation and/or problem solving 7.) Engaging and visual presentations 8.) Resolving conflicts |

| Not For | 1.) Time-sensitive updates 2.) Internal communication 4.) Team-specific news 5.)

Daily operational matters | 1.) Technical documentation 2.) Small audience communication 3.)

Highly confidential information. Note: Private Sites is an option for safeguarding access | 1.)

Detailed documentation 2.) Complex content sharing 3.) Evergreen or long-term Information

sharing | 1.) Collaborative discussions 2.) Questions requiring quick responses 3.) Urgent

issues requiring immediate attention | 1.) Simple updates 2.) Information sharing only 3.)

Documentation of processes 4.) That which could be handled asynchronously |

| Key Features / Benefits | 1.) Version control 2.) Public accessibility 3.) Single source of

truth for processes | 1.) Customization and personalization 2.) Newsletter, feed, employee

generated content 3.) AI powered search and content governance 4.) Fosters Communication,

connection, and culture | 1.) Collaboration and connection 2.) Integrations 3.) Real-time

communication 4.) Productivity and efficiency 5.) Supports projects, teams, and transparency |

1.) Improved awareness 2.) Streamlined communication 3.) Personalized content delivery | 1.)

Face-to-face interaction 2.)Screen and presentation sharing 3.) Breakout rooms 4.) Recording

capability |

Structure content instead of using FAQs

We want to avoid unstructured content which includes FAQs (Frequently Asked Questions),

especially for internal communication.

FAQs tend to take on the voice and concerns of assumed personas. Instead of assuming questions, aim to articulate key facts as statements and use these to structure content under topical headers which aren't questions.

Structured content around GitLab, the product, should be in GitLab Docs and structured content around GitLab, the company, should be in the handbook; we should not use separate documents or locations to share this information.

Restructuring the content

As an example, let's say your FAQ would have a question like:

> Q: I'm not seeing widget X, what should I do?

> A: If you're not seeing widget X, you can verify if it's enabled or not by going to User Profile -> Settings and ensure the checkbox is enabled under Enable widget X

You can reframe it to:

How to enable widget X

> You can enable widget X by going to User Profile -> Settings and ticking the checkbox next to Enable widget X then clicking on the Save button

FAQs are discouraged elsewhere as well

Content guidelines across the industry support avoiding FAQs:

1. <https://www.plainlanguage.gov/guidelines/web/avoid-faqs/>
1. <https://digital.gov/2015/04/27/are-faqs-still-relevant/>
1. <https://guides.18f.gov/content-guide/our-approach/structure-the-content/>
1. <https://gds.blog.gov.uk/2013/07/25/faqs-why-we-dont-have-them/>
1. <https://thegood.com/insights/faq-pages/>
1. <https://alistapart.com/article/no-more-faqs-create-purposeful-information-for-a-more-effective-user-experi/>

Multimodal communication

Employ multimodal communication to broadcast important decisions. To reach our distributed organization, announce important decisions in the company announcements Slack channel, email the appropriate team email lists, Slack the appropriate channels, and target 1:1s or other important meetings on the same day, with the same information.

When doing this, create and link to a single source of truth: ideally the handbook, otherwise an epic, issue, or Google Doc. The email or Slack message should not be the source of truth.

When referring to email that recipients should have received, reference the sender and subject of the email so it's easy to find. For example, "You should have received an email from Jane Smith with the subject 'Training Seminar Details'".

Asking "is this known"

If something is behaving strangely on <https://gitlab.com>, it might be a bug. It could also mean that something was changed intentionally.

Please search if the issue has already been reported. If it has not been reported, and you are sure it is a bug, please file an issue.

If you are unsure whether the behavior you experience is a bug, you may ask in the Slack channel `is-this-known`.

If you know which stage of the DevOps lifecycle is affected, it is also okay to ask in `s\{stage}`, for example `smanage`.

When you ask:

- Make sure that no-one has experienced this issue before, by checking the channel for previous messages.

- Describe the behavior you are experiencing, this makes it searchable and easier to understand.

Different people might look for different things in the same screenshots.

- Asking in a single channel helps discoverability, duplicated efforts and reduces noise in other channels.

Please refrain from asking in general purpose channels like frontend, backend, development or questions.

Numbering is for reference, not as a signal

When taking notes in an agenda, in the handbook, or on our OKRs, keep items numbered so we can refer to Item 3 or 4a.

The number is not a signal of the importance or rank of the subject unless explicitly stated to be such.

It is just for ease of reference.

Cross link

Linking should not be in one direction. We should go beyond deep-linking to create a richer web of links that can surface content and ensure people consider all pages when making updates. When linking one page to another, try to link back as well. Instead of only linking from Page A to Page B, both link Page A to Page B and link Page B back to Page A. For example, the Live Doc Meeting section of the All Remote Guide links to the Live Docs Meetings page. The Live Docs Meetings page links back to the Live Doc Meeting section of the All Remote Guide.

Acknowledgement Receipts (ACK)

Informal ACKs

In informal acknowledgement scenarios, such as on Slack or on issue comments, it is common practice to use the following:

- Slack emoji reaction of `:ack:` or an ACK response => Acknowledged, or message received

- Eyes · => I'll check this out | seen | working on it
- Thumbs up · => good idea
- White checkmark · => task is complete or done
- Heart · = expression of gratitude or appreciation
- cc @mentions => if someone needs to see a message

Formal ACKs

In order to effectively communicate an important change to hundreds of distributed employees, we occasionally use an ACK process.

To prevent overuse, this should only be used by a member of the exec team. Anyone may ask an exec to sponsor one.

As a guideline, we'd expect no more than one per quarter to be sent out. Too many ACKs lose power.

To initiate an ACK process:

1. Clone the form from the ACK template and fill it out.
 1. Link to MRs and Handbook pages instead of duplicating your content in the form. Why handbook first?
 1. Ask People Ops to pull a report from Workday with the column headers First Name, Last Name, Job Title, Department, Manager, and Work Email. Double check it and turn the emails into a comma-delimited string with an excel formula like this: =TEXTJOIN(", ", true, Sheet1!E2:E432)
 1. Send the form and expect to get 50% of the responses in the first 24 hours. To get the rest:
 1. Post in common Slack channels.
 1. Add to staff meeting agendas.
 1. Suggest to team managers to post to their team Slack channels, ask for explicit :ack: and pin to the channel until everyone responds.
 1. Lastly, reach out 1-on-1 to stragglers while being respectful of vacation time.

Say thanks

As we continue to build on inclusion, recognition is a key and transformative tactic. Thanking team members provides an opportunity for them to be recognized for their contributions, influences engagement behavior, and acknowledges to team members their work is seen. By saying thanks, you are contributing to and supporting the value of DIB.

1. Thank people that did a great job in our thanks Slack channel. Almost everyone in the company is active in this channel so please don't be shy.
 1. Consider other channels where recognition can be acknowledged: team meetings, issues, company calls, 1-1 meetings, etc.
 1. If someone is a team member just @-mention them, if multiple people were working on something try @-mentioning each person.
 1. When announcing a completed project, list the key contributors.
 1. Please be as timely as possible with your recognition.
 1. If possible please include a link with your thanks that points to the subject matter that you are giving thanks for, for example a link to a merge request.
 1. Please do not mention working outside of working hours, we want to minimize the pressure to

do so.

1. Please do not celebrate GitLab contribution graphs that include working for uninterrupted weeklong cycles, as this does not foster healthy work/life harmony for all team members. While GitLab team members are free to time-shift and work weekends in place of weekdays, we discourage celebrating the absence of time away from work.

1. Do not thank the CEO or other executives for something that the company paid for, thank GitLab instead.

1. To thank someone who is not a team member, you can nominate them for community swag.

1. Understand that everyone doesn't need or want recognition. Once this is advised, please respect when they don't.

Values e